

Projektstatusbericht

Gruppe 7 – Datenmanagement und Analytics (M.Sc.) SoSe 26

Banana Supply Chain Datenplattform

Erstellt am:	2026-06-02
Deadline:	01.07.2026
Modul:	Datenmanagement und Analytics (M.Sc.), SoSe 26
Repository:	git.mylab.th-luebeck.de/omied.firouzian/gruppe7_dma_rose26
Letzte Commits:	V3 – alle Daten hochgeladen (2026-05-13)

Gesamtstatus Teil 1: Alle Pflichtanforderungen erfüllt ■

Dieser Bericht fasst alle erledigten Aufgaben, gelieferten Artefakte und noch offenen Punkte für das Projekt Banana Supply Chain Datenplattform zusammen.

1. Erledigte Aufgaben – Teil 1: Datenmanagement

1.1 Infrastruktur

#	Anforderung	Status	Nachweis
I-1	Gruppe & Repo eingerichtet	■	GitLab-Repo: gruppe7_dma_sose26
I-2	Vorgegebene Folderstruktur hochgeladen	■	bananasupplychain/, databasemodels_logistics_playground/
I-3	Docker Container startbar	■	docker-compose.yml – 5 Services: PostgreSQL, MongoDB, Redis, Neo4j
I-4	Datengenerator ausgeführt	■	shared/erp/ (50), shared/wms/ (70), shared/tms/ (263) JSON-Dateien

1.2 Datenklassifikation

#	Anforderung	Status	Nachweis
D-1	JSON-Dateien nach Stamm-/Bewegungsdaten klassifiziert	■	docs/01_data_classification.md – 13 Eventtypen
D-2	JSON-Dateien nach Zieldatenbanken klassifiziert	■	Tabelle mit Primär- und Sekundär-Zielsystem je Event

1.3 PostgreSQL Datenmodelle

#	Anforderung	Status	Nachweis
P-1	Datenmodell ERP mit ER-Modell	■	sql/02_create_erp_tables.sql – 6 Tabellen (suppliers, customers, products)
P-2	Datenmodell WMS mit ER-Modell	■	sql/03_create_wms_tables.sql – 3 Tabellen (warehouse_skus, supply_chain_nodes, carriers)
P-3	Datenmodell TMS mit ER-Modell	■	sql/04_create_tms_tables.sql – 6 Tabellen (carriers, transport_product_requests, transport_product_assignments)
P-4	ER-Modell mit Kardinalitäten (Mermaid)	■	docs/03_er_model.md – vollständiges ER-Diagramm mit PKs, FKs, Kardinalitäten

1.4 Masterdatenmanagement (MDM)

#	Anforderung	Status	Nachweis
M-1	MDM-Schema entwickelt	■	sql/05_create_mdm_tables.sql – entity_types, golden_records, source_systems
M-2	Inkonsistenz BAN-101 / BAN_101 / ban-101 adressiert	■	docs/04_masterdata_management.md – mdm.resolve_canonical_key()
M-3	Alle relevanten Entitäten im MDM	■	Produkte, Lieferanten, Kunden, Carrier, Supply-Chain-Knoten

1.5 Metadatenmanagement

#	Anforderung	Status	Nachweis
Me-1	Metadaten-Schema entwickelt	■	sql/06_create_metadata_tables.sql – systems, tables, columns
Me-2	Skalenniveaus für alle Spalten bestimmt	■	docs/05_metadata_management.md – NOMINAL/ORDINAL/INTERVAL
Me-3	Qualitätsregeln in Metadaten hinterlegt	■	meta.columns.quality_rule für alle wichtigen Spalten

#	Anforderung	Status	Nachweis
Me-4	Skalenniveaus fachlich begründet	■	temperature (INTERVAL), delay_minutes (RATIO), delivery_priority (OR

1.6 Data Warehouse

#	Anforderung	Status	Nachweis
DW-1	DWH-Schema (Sternschema) entwickelt	■	sql/07_create_dwh_schema.sql – 7 Dimensionen + 1 Faktentabelle
DW-2	ETL-Prozess ERP/WMS/TMS → DWH dokumentiert	■	docs/07_dwh_model.md Kap. 5 + docs/12_etl_concept.md Phase 2
DW-3	Klare Trennung operative Schemas ≠ DWH	■	Separates dwh-Schema, nur ETL-Schreibzugriff
DW-4	Kennzahlen (Measures) definiert	■	quantity, unit_price, total_value, delay_minutes, avg_temperature, num_

1.7 Neo4j Graphmodell

#	Anforderung	Status	Nachweis
N-1	Neo4j Instanz modelliert	■	docs/10_neo4j_graph_model.md + cypher/01_create_graph_model.cyp
N-2	Nodes und Beziehungen definiert	■	8 Node-Typen, 12 Relationship-Typen
N-3	Begründung für Graphdatenbank	■	SQL vs. Cypher Vergleich für tiefe Pfad-Abfragen
N-4	Beispiel-Cypher-Abfragen	■	5 Abfragen inkl. Supply-Chain-Pfad PLANTATION→RETAIL

1.8 MongoDB Eventmodell

#	Anforderung	Status	Nachweis
Mo-1	MongoDB Instanz modelliert	■	docs/08_mongodb_event_model.md
Mo-2	Collections für Events, Tracking, Nodes	■	4 Collections: shipment_events, node_events, batch_tracking, order_events
Mo-3	Beispieldokumente und Indizes definiert	■	Vollständige JSON-Beispiele mit Index-Definitionen + TTL-Index
Mo-4	Begründung für MongoDB	■	Flexible Schemas, hohe Schreibleistung, embedded Documents

1.9 MinIO Dokumentenspeicher

#	Anforderung	Status	Nachweis
Mi-1	MinIO Instanz für Dokumente modelliert	■	docs/11_minio_document_model.md
Mi-2	Bucket-Struktur definiert	■	4 Buckets: invoices, delivery-notes, transport-docs, batch-certificates
Mi-3	Begründung Object Store vs. Datenbank	■	BLOBs in DB verlangsamen Backups; S3-kompatible URLs; Horizontal s
Mi-4	Referenzierungsmuster PostgreSQL ↔ MinIO	■	Dokumentreferenz-Tabelle mit Bucket/Objektpfad in PostgreSQL

1.10 Redis Echtzeitmodell

#	Anforderung	Status	Nachweis
R-1	Redis Instanz für Echtzeitdaten modelliert	■	docs/09_redis_realtime_model.md
R-2	Key-Struktur definiert	■	STRING, HASH, LIST, SORTED SET, COUNTER mit TTL
R-3	Begründung für Redis	■	Sub-Millisekunden-Latenz für Shipment-Tracking und GPS-Updates

1.11 Datenqualitätsmanagement

#	Anforderung	Status	Nachweis
Q-1	Konkrete DQ-Regeln definiert	■	docs/06_data_quality.md – 6 Dimensionen mit je 2–4 Regeln
Q-2	Bezug auf Supply-Chain-Beispiele	■	Kühlkette (PQ-03), Produktcode-Harmonisierung (KQ-01), Zeitlogik (AQ
Q-3	SQL-Checks implementiert	■	sql/08_data_quality_checks.sql – 20+ Checks über alle 6 Dimensionen
Q-4	Python-Validierungsfunktionen	■	docs/06_data_quality.md Kap. 4 + docs/12_etl_concept.md

1.12 ETL-Konzept und Implementierung

#	Anforderung	Status	Nachweis
E-1	ETL-Konzept für alle Zielsysteme	■	docs/12_etl_concept.md – Extract, Transform, Load für alle 5 Systeme
E-2	Mapping-Tabelle (Quelle → Transformation → Ziel)	■	docs/12_etl_concept.md Kap. 4 – alle 13 Eventtypen gemappt
E-3	ERP/WMS/TMS als operative Quellsysteme klar definiert	■	docs/12_etl_concept.md Kap. 3 + Architekturdiagramm
E-4	ETL Phase 2: operative Schemas → DWH	■	docs/12_etl_concept.md Kap. 3 mit SQL-ETL-Beispiel

2. Gelieferte Artefakte

Ordner/Datei	Inhalt	Anzahl	Status
shared/erp/	JSON-Events (Supplier, Customer, Product, Order, Batch)	500 Dateien	■
shared/wms/	JSON-Events (WarehouseSKU, NodeProcessed)	70 Dateien	■
shared/tms/	JSON-Events (Carrier, Transport, GPS, Delivery)	263 Dateien	■
docs/	Markdown-Dokumentation (Architektur, Modelle, Konfiguration)	18 Dateien	■
sql/	PostgreSQL DDL-Skripte (Schemas, Tabellen, DWH-Dimensionen)	8 Dateien	■
cypher/	Neo4j Cypher-Skript (Constraints, Stammdaten, Topologie)	1 Datei	■
bananasupplychain/etl_load.py	ETL-Skript – lädt 383 Events in 5 Systeme	1 Datei	■
bananasupplychain/generate_documents.py	MinIO Dokumentengenerator (PDFs + PostgreSQL-Referenzen)	1 Datei	■
bananasupplychain/container/docker-compose.yml	Docker-Setup (PostgreSQL, MongoDB, Redis, Neo4j, MinIO)	1 Datei	■

2.1 ETL-Testergebnisse (getestet 2026-05-12)

Alle Komponenten wurden gegen die laufenden Docker-Container getestet:

Komponente	Test-Ergebnis	Details
PostgreSQL SQL 01–08	■ Alle ausgeführt	6 Schemas, 26 Tabellen inkl. erp.document_references
PostgreSQL MDM-Funktion	■ Funktioniert	BAN_101 / ban-101 / BAN-101 → alle → BAN-101
PostgreSQL DWH dim_date	■ 1095 Zeilen	2025-01-01 bis 2027-12-31
MongoDB Collections	■ 4 Collections	shipment_events (500), node_events (121), batch_tracking (11), order_events (11)
Redis Key-Typen	■ Alle funktionieren	STRING, HASH, LIST, SORTED SET, COUNTER
Neo4j Graphmodell	■ 125 Nodes, 47+ Relations	Supply-Chain-Pfad PLANTATION→RETAIL in 6 Hops
MinIO Buckets + Dokumente	■ 4 Buckets	66 Dokument-Referenzen in PostgreSQL
ETL-Skript (etl_load.py)	■ Vollständig	383 Events erfolgreich in alle 5 Systeme geladen

2.2 Daten in den Systemen nach ETL

System	Einträge geladen
PostgreSQL	10 Supplier, 10 Customer, 10 Products, 10 Orders, 10 Batches, 60 Shipments, 118 Positions, 60 Completions, 10 Deliveries
MongoDB	500 shipment_events, 121 node_events, 11 batch_tracking, 11 order_events
Redis	60 Shipment-Status, 118 Position-Updates, 10 Delivery-Status
Neo4j	61 Shipments, 11 Orders, 11 Batches + Stammdaten (Supplier, Customer, Carrier, Nodes)
MinIO	60 Lieferscheine, 6 Rechnungen (66 Referenzen in PostgreSQL)

3. Offene Punkte und Verbesserungspotenzial

3.1 Pflichtaufgaben – Teil 2: Analytics (Deadline: 01.07.2026)

Teil 2 des Projekts ist noch vollständig offen. Folgende Aufgaben müssen bis zur Abgabe erledigt werden:

#	Aufgabe	Beschreibung	Prio
A-1	Deskriptive Statistik	Kennzahlen aus dem DWH berechnen (Mittelwert, Median, Standardabweichung, Min/Max für delay_m)	Hoch
A-2	KPI-Definition	Mindestens 5 Business-KPIs definieren (z.B. Liefertreue, Ø Transportkosten, Temperaturausreißer-Quo)	Hoch
A-3	Python-Charts (5 Stück)	Matplotlib/Seaborn-Visualisierungen: Histogramme, Boxplots, Zeitreihenplots, Heatmaps o.ä.	Hoch
A-4	PowerBI-Dashboard	Mindestens 1 Dashboard mit mehreren Visuals auf Basis der DWH-Daten	Hoch
A-5	Clusteranalyse	Kundensegmentierung oder Routen-Cluster (z.B. k-Means auf Shipments-Daten)	Mittel
A-6	Prognosemodell	Zeitreihenprognose (z.B. ARIMA oder Prophet für Liefervolumen oder Verzögerungen)	Mittel
A-7	Abschlussbericht	Zusammenfassung aller Ergebnisse in einem Bericht (Markdown oder PDF)	Hoch

3.2 Verbesserungspotenzial – Teil 1 (optional, aber empfohlen)

#	Bereich	Was fehlt / was könnte besser sein	Aufwand
V-1	ETL-Skript	Fehlerbehandlung: Falls ein JSON-Event ein unbekanntes Format hat, bricht das Skript ab. Try/Except-Blöcke	Niedrig
V-2	ETL-Skript	Idempotenz: Mehrfaches Ausführen von etl_load.py erzeugt Duplikate in PostgreSQL. Ein ON CONFLICT DO	Niedrig
V-3	DWH	ETL Phase 2 (operative Schemas → DWH) ist konzipiert aber noch nicht als vollfähiges Skript implementiert. M	Mittel
V-4	Datenqualität	Die DQ-Checks in sql/08_data_quality_checks.sql laufen manuell. Eine automatisierte Ausführung nach dem E	Mittel
V-5	Neo4j	Der ETL-Loader befüllt Neo4j nur mit Stammdaten. Tatsächliche Fulfillment-Events aus den TMS-Daten werde	Niedrig
V-6	Dokumentation	Die Mermaid-Diagramme in den Markdown-Dateien können nicht direkt in PDF gerendert werden – für Abgab	Niedrig
V-7	Git-Workflow	VS Code erzeugt regelmäßig index.lock-Dateien beim gleichzeitigen Commit. Empfehlung: Git-Operationen	Niedrig

4. Zeitplan bis Abgabe (01.07.2026)

Zeitraum	Aufgabe	Status
Bis 13.05.2026	Teil 1: Infrastruktur, Datenmodelle, ETL, Dokumentation	■ Abgeschlossen
Mai 2026	Verbesserungen Teil 1 (ETL-Idempotenz, DQ-Automatisierung)	■ Optional
Jun 2026 – Woche 1	Deskriptive Statistik & KPI-Berechnung (Python)	■ Offen
Jun 2026 – Woche 1	5 Python-Charts erstellen (Matplotlib/Seaborn)	■ Offen
Jun 2026 – Woche 2	PowerBI-Dashboard aufbauen	■ Offen
Jun 2026 – Woche 3	Clusteranalyse und Prognosemodell	■ Offen
Jun 2026 – Woche 4	Abschlussbericht schreiben & finaler Commit	■ Offen
01.07.2026	Deadline – Abgabe	■ Frist

5. Zusammenfassung

■ Komplette erledigt (Teil 1)

- Infrastruktur: Docker-Setup mit 5 Datenbanken läuft
- Datengenerierung: 383 JSON-Events (ERP 50, WMS 70, TMS 263)
- PostgreSQL: 26 Tabellen in 6 Schemas (ERP, WMS, TMS, MDM, Meta, DWH)
- ETL-Skript: Alle 383 Events in alle 5 Zielsysteme geladen
- Neo4j: 8 Node-Typen, 12 Relationship-Typen, Cypher-Abfragen
- MongoDB: 4 Collections, 643 Dokumente
- Redis: Vollständige Key-Taxonomie mit TTL
- MinIO: 4 Buckets, 66 Dokumentreferenzen in PostgreSQL
- Dokumentation: 13 Markdown-Dateien, vollständig

■ Noch offen (Teil 2 – Analytics)

- Deskriptive Statistik und KPI-Berechnung
- 5 Python-Visualisierungen (Matplotlib/Seaborn)
- PowerBI-Dashboard
- Clusteranalyse (z.B. Kundensegmentierung)
- Prognosemodell (Zeitreihe)
- Abschlussbericht

■■ Empfohlene Verbesserungen

- ETL-Idempotenz: ON CONFLICT DO NOTHING in etl_load.py
- DQ-Automatisierung: DQ-Checks nach ETL automatisch ausführen
- DWH ETL Phase 2: als lauffähiges Python-Skript implementieren
- Neo4j ETL: Fulfillment-Routen automatisch aus TMS-Daten laden

